

## Code sample



**Note:** You need to be running the latest kernel to make this program work..

This sample program, `my_debugfs.c`, shows a use of DebugFS.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/debugfs.h> /* this is for DebugFS libraries */
#include <linux/fs.h>
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Surya Prabhakar <surya_prabhakar@dell.com>");
MODULE_DESCRIPTION("sample code for DebugFS functionality");
#define len 256
u64 intvalue, hexvalue;
struct dentry *dirret, *fileret, *u64int, *u64hex;
char ker_buf[len];
int filevalue;
/* read file operation */
static ssize_t myreader(struct file *fp, char __user *user_buffer,
                       size_t count, loff_t *position)
{
    return simple_read_from_buffer(user_buffer, count, position, ker_buf, len);
}

/* write file operation */
static ssize_t mywriter(struct file *fp, const char __user *user_buffer,
                      size_t count, loff_t *position)
{
    if(count > len)
        return -EINVAL;

    return simple_write_to_buffer(ker_buf, len, position, user_buffer, count);
}

static const struct file_operations fops_debug = {
    .read = myreader,
    .write = mywriter,
};

static int __init init_debug(void)
{
    /* create a directory by the name dell in /sys/kernel/debugfs */
    dirret = debugfs_create_dir("dell", NULL);

    /* create a file in the above directory
    This requires read and write file operations */
    fileret = debugfs_create_file("text", 0644, dirret, &filevalue, &fops_debug);

    /* create a file which takes in a int(64) value */
    u64int = debugfs_create_u64("number", 0644, dirret, &intvalue);
```

```
if (!u64int) {
    printk("error creating int file");
    return (-EINVAL);
}

/* takes a hex decimal value */
u64hex = debugfs_create_u64("hexnum", 0644, dirret, &hexvalue);
if (!u64hex) {
    printk("error creating hex file");
    return (-EINVAL);
}

return (0);
}

module_init(init_debug);

static void __exit exit_debug(void)
{
    /* removing the directory recursively which
    in turn cleans all the file */
    debugfs_remove_recursive(dirret);
}

module_exit(exit_debug);
```

## Building the module

Create a makefile like the following one, to compile and test this code:

```
obj-m := my_debugfs.o
KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)
```

all:

```
$(MAKE) -C $(KDIR) SUBDIRS=$(PWD) modules
```



## Further reading and references

For those who are unaware, there's lot of information already available; Linux Weekly News and the LKML mailing list are good sources of knowledge. If you want to look at the relevant kernel source code, you can visit:

<http://lxr.linux.no/#linux+v2.6.35/include/linux/debugfs.h>

<http://lxr.linux.no/#linux+v2.6.35/fs/debugfs/inode.c>

You can observe the latest changes in DebugFS by examining the online Git repository at kernel.org:

<http://git.kernel.org/?p=linux/kernel/git/next/linux-next.git;a=history;f=include/linux/debugfs.h;h=e7d9b20ddc5b98c6e5ae6b89ee2a2b72716d9e76;hb=HEAD>

Within the kernel source, you can refer to Documentation/filesystems/debugfs.txt.

## By: Surya Prabhakar

The author is an engineering advisor in the Product Group at Dell India R&D centre, Bengaluru and has 8 years of experience in Linux. He spends most of his time hacking and playing with Linux. He can be reached at [surya\\_prabhakar@dell.com](mailto:surya_prabhakar@dell.com)